

Lecture 7 & 8

Web Programming



Async/Eventloops

DOM

Jquery

Mahhek Tahir



Async/Eventloops



Asynchronous

JavaScript is **single-threaded**, meaning it executes one task at a time. However, **asynchronous programming** allows it to handle multiple tasks **without blocking execution**.

Synchronous Code

(Executes One Task at a Time)

javascript

Copy Edit

```
console.log("Start");  
console.log("Middle");  
console.log("End");
```

📌 Output:

sql

Copy Edit

```
Start  
Middle  
End
```

✅ Executes in order, blocking the next task until the current one finishes.



Async/await

Because some tasks take time:

- API calls
- Database fetch
- File loading
- Geolocation
- Timers



API Call (fetch data from server)

```
async function getUsers() {  
  const res = await fetch("https://jsonplaceholder.typicode.com/users");  
  const data = await res.json();  
  console.log(data);  
}  
  
getUsers();
```

wait for api to return, get json and then print it

File Loading (read file content)

```
async function loadFile() {  
    const res = await fetch("sample.txt");  
    const text = await res.text();  
    console.log(text);  
}  
  
loadFile();
```

Fetch api using JS



```
// Example function to fetch data from an API
async function fetchData(url) {
  try {
    // Making a GET request to the API
    const response = await fetch(url);

    // Checking if the response is okay (status code 200-299)
    if (!response.ok) {
      throw new Error('Network response was not ok');
    }

    // Parsing the response as JSON
    const data = await response.json();
    console.log(data); // Logging the data from the API
  } catch (error) {
    console.error('There was a problem with the fetch operation:', error);
  }
}

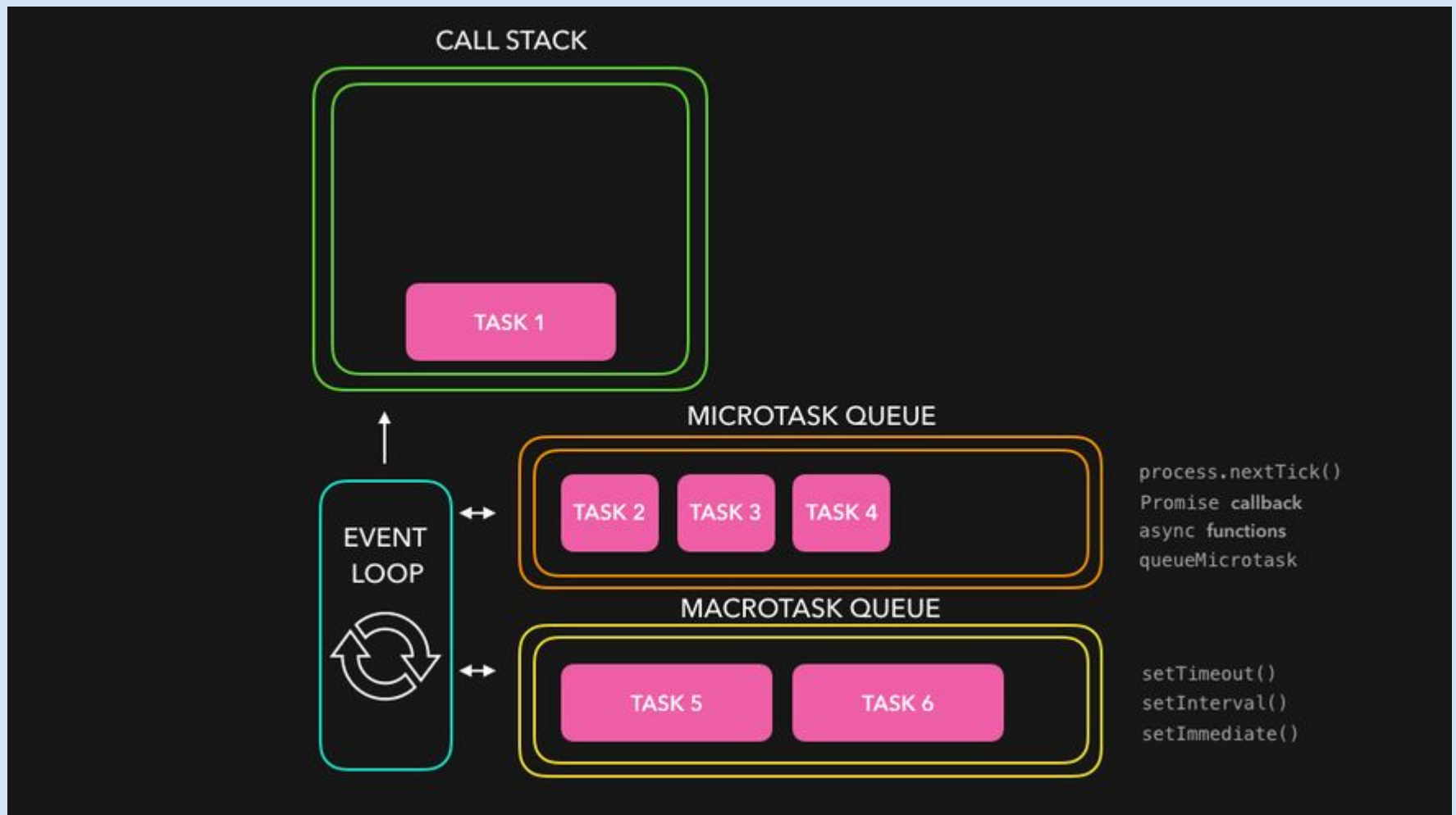
// Example API URL (Replace with your actual API URL)
const apiUrl = 'https://jsonplaceholder.typicode.com/todos/1';
fetchData(apiUrl);
```



Event Loop

- ▶ The **Event Loop** is what makes JavaScript **asynchronous**. It allows JavaScript to handle **non-blocking** operations, even though it's single-threaded.

Event Loop

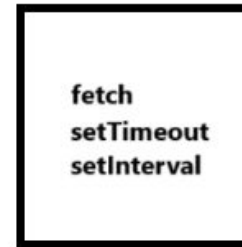


Event Loop

```
function main ( ){  
  console.log("Start");  
  
  setTimeout(() => {  
    console.log("Async operation");  
  }, 2000);  
  
  console.log("End");  
}  
main();
```



Call Stack



Web APIs

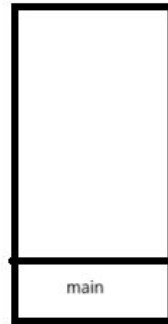


queue

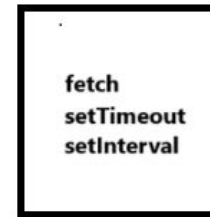
Console Output:

Event Loop

```
function main () {  
  console.log('Start');  
  
  setTimeout(() => {  
    console.log('Async operation');  
  }, 2000);  
  
  console.log('End');  
}  
main();
```



Call Stack



Web APIs

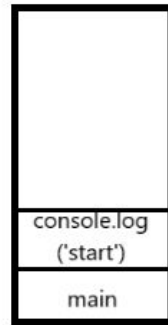


queue

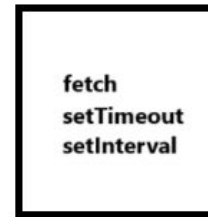
Console Output:

Event Loop

```
function main () {  
  console.log('Start');  
  
  setTimeout(() => {  
    console.log('Async operation');  
  }, 2000);  
  
  console.log('End');  
}  
main();
```



Call Stack



Web APIs



queue

Console Output:

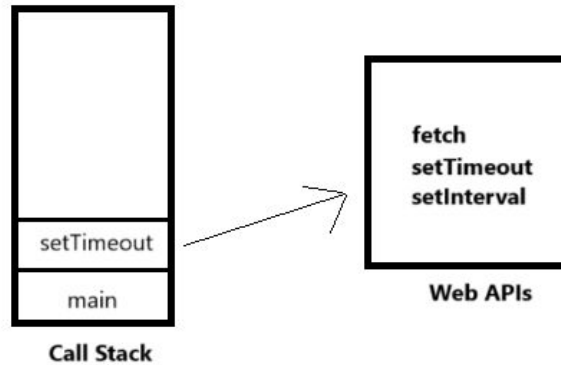
start

Event Loop

```
function main () {
  console.log('Start');

  setTimeout(() => {
    console.log('Async operation');
  }, 2000);

  console.log('End');
}
main();
```

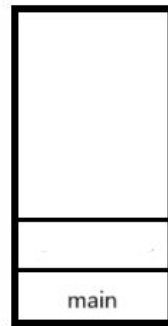


Console Output:

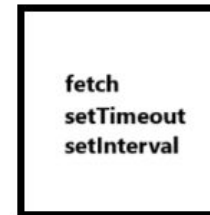
start

Event Loop

```
function main () {  
  console.log('Start');  
  
  setTimeout(() => {  
    console.log('Async operation');  
  }, 2000);  
  
  console.log('End');  
}  
main();
```



Call Stack



Web APIs

callback in the setTimeout
placed in queue when the timer
expires



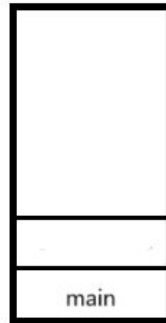
queue

Console Output:

start

Event Loop

```
function main () {  
  console.log('Start');  
  
  setTimeout(() => {  
    console.log('Async operation');  
  }, 2000);  
  
  console.log('End');  
}  
main();
```



Call Stack



Web APIs



queue

Console Output:

start
End

Event Loop

```
function main () {
  console.log('Start');

  setTimeout(() => {
    console.log('Async operation');
  }, 2000);

  console.log('End');
}
main();
```



Call Stack



Web APIs



Console Output:

start
End

Event Loop

```
function main(){  
  console.log('Start');  
  
  setTimeout(() => {  
    console.log('Async operation');  
  }, 2000);  
  
  console.log('End');  
}  
main();
```



Call Stack



Web APIs

call stack was empty, so the event loop placed the callback on the call stack.



queue

Console Output:

start
End

Event Loop

```
function main(){
  console.log('Start');

  setTimeout(() => {
    console.log('Async operation');
  }, 2000);

  console.log('End');
}
main();
```



Call Stack



Web APIs



queue

Console Output:

```
start
End
Async Operation
```



**Code after
`await` runs
in the
Microtask
queue. If
called
directly
without
`await`, it
runs
immediately
on the Call
Stack.**

Stack : [console.log]

Micro : []

Macro : []

Output: Start

Stack : []

Micro : []

Macro : [Timer]

Output: Start

Stack : []

Micro : [Async Done]

Macro : [Timer]

Output: Start, Hi

Stack : [greet → console.log]

Micro : []

Macro : [Timer]

Output: Start, Hi

```
console.log("Start");
```

```
setTimeout(() => console.log("Timer"), 0);
```

```
async function greet() {  
  console.log("Hi");  
  await Promise.resolve();  
  console.log("Async Done");  
}
```

```
greet();
```

```
console.log("End");
```

Stack : [Async Done]

Micro : []

Macro : [Timer]

Output: Start, Hi, End, Async Done

Stack : [Timer]

Micro : []

Macro : []

Output: Start, Hi, End, Async Done, Timer

Stack : [console.log]

Micro : [Async Done]

Macro : [Timer]

Output: Start, Hi, End



Document Object Model



Document Object Model - DOM

- ▶ A programming interface for web documents
- ▶ Represents the HTML structure as a tree
- ▶ Enables JavaScript to interact with web pages dynamically



DOM Structure

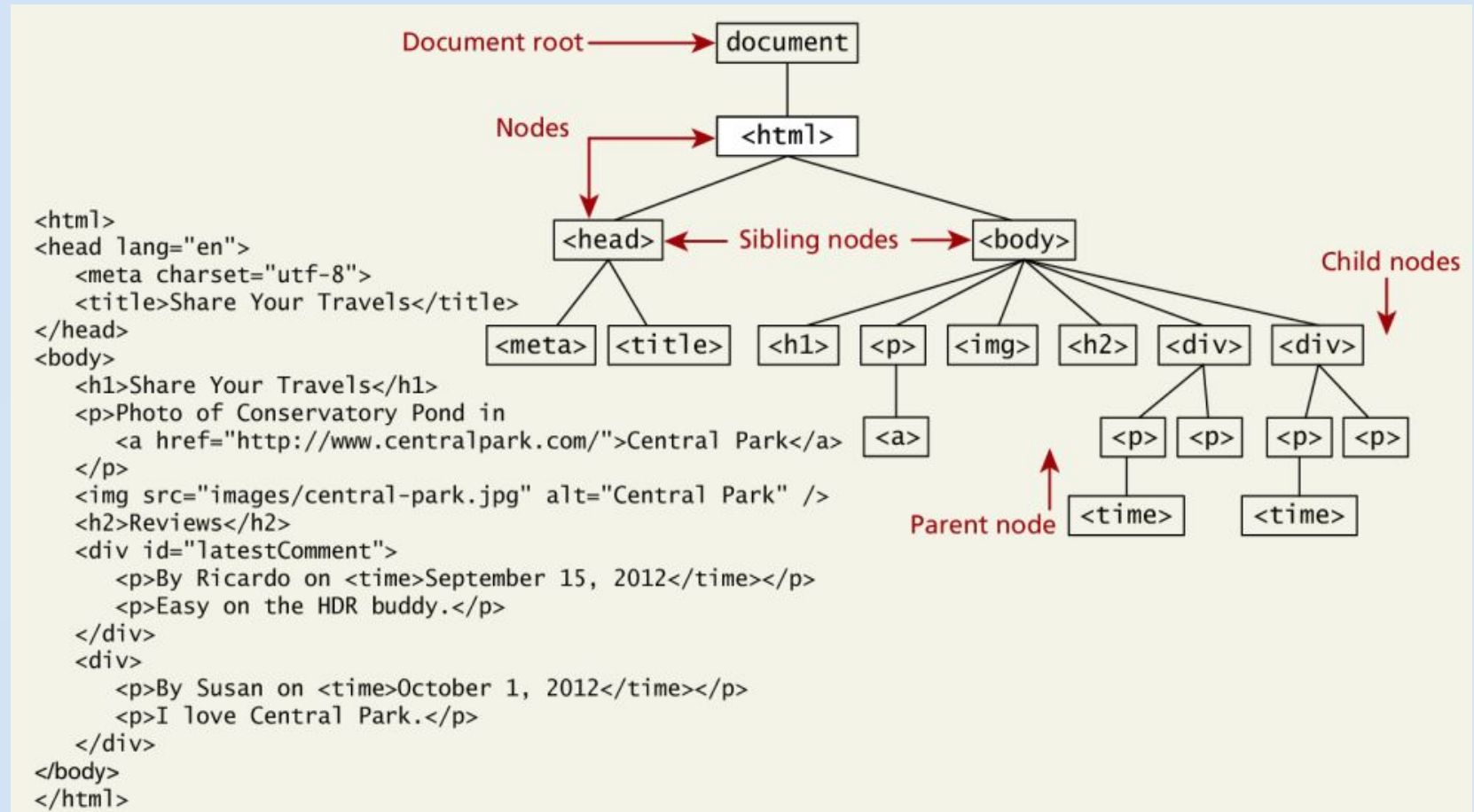
- The DOM is structured like a tree
- Root Node → Document
- Element Nodes → HTML elements
- Attribute Nodes → Attributes inside elements
- Text Nodes → Text inside elements



How DOM Works

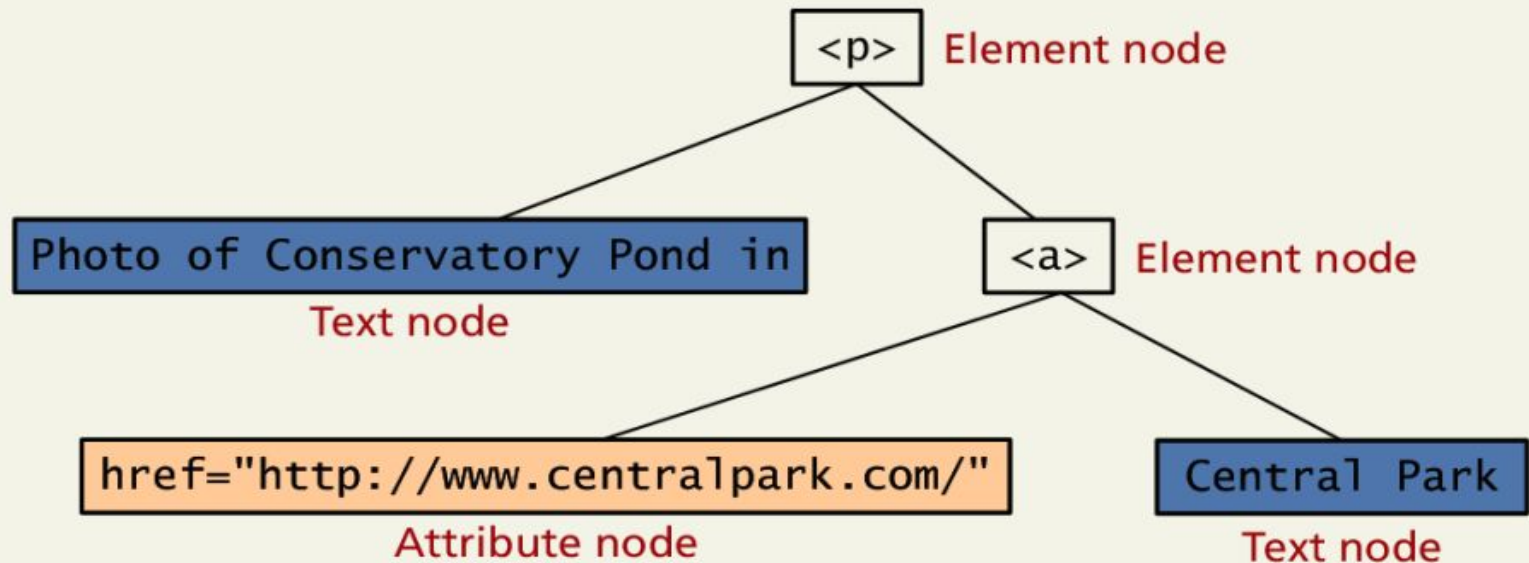
- Browser parses an HTML document and creates a DOM tree
- JavaScript can modify this tree dynamically
- The DOM is an in-memory representation of the document

DOM Tree



DOM Node

```
<p>Photo of Conservatory Pond in  
  <a href="http://www.centralpark.com/">Central Park</a>  
</p>
```





Accessing DOM Elements

- `document.getElementById('id')`
- `document.getElementsByClassName('class')`
- `document.querySelector('selector')` //only first match
- `document.querySelectorAll('selector')`
- `document.getElementsByTagName("tagName")`



Modifying the DOM

- Change content: `element.innerHTML = '<div> new element</div>'`
- Change text: `element.textContent = "New Text";`
- Change styles: `element.style.color = 'blue';`
- Can be used to get and set `innerHTML`, `textContent` and `style`
- `element.innerHTML` will return inner Html of the element and `element.innerHTML = ""` will set it



Getting the first and last element

First Element

// Get all <p> elements

```
let paragraphs = document.getElementsByTagName("p");
```

// Select the first one

```
let firstParagraph = paragraphs[0];
```

Last Element

// Get all <p> elements

```
let paragraphs = document.getElementsByTagName("p");
```

// Select the last one

```
let lastParagraph = paragraphs[paragraphs.length - 1];
```

Node



```
<div id="container">
  <h2 id="title">My List</h2>

  <ul id="list">
    <li id="item1">Apple</li>
    <li id="item2">Banana</li>
    <li id="item3">Mango</li>
  </ul>

  <p id="note">End of list</p>
</div>
```

```
document.getElementById("item2").parentElement → <ul id="list">
document.getElementById("item2").nextElementSibling → <li id="item3">Mango</li>
document.getElementById("item2").previousElementSibling → <li id="item1">Apple</li>
document.getElementById("item2").children → HTMLCollection(0) []
document.getElementById("item2").textContent → "Banana"
document.getElementById("list").firstElementChild → <li id="item1">Apple</li>
document.getElementById("list").lastElementChild → <li id="item3">Mango</li>
```

textContent

text content will return all the children node text,
Text before First Child Second Child Text after

```
<div id="parent">  
  Text before  
  <p id="child1">First Child</p>  
  <p id="child2">Second Child</p>  
  Text after  
</div>
```



Form Validation

```
<html>
  <head>
    <script type="text/javascript">
      function validate(){
        if (document.getElementById('q').value == ""){
          alert('search string cannot be empty');
          return false;
        }
        return true;
      }
    </script>
  </head>
  <body>
    <form method="GET" action="http://www.google.com/search">
      <input type="text" name="q" id="q" />
      <input type="submit" value="Google Search" onclick="return validate()"/>
    </form>
  </body>
</html>
```



getAttribute() vs setAttribute

```
<button id="btn1" name="submitBtn" value="Send">
```

Click Me

```
</button>
```

```
const button = document.getElementById("btn1");
```

```
console.log(button.getAttribute("id"));    // btn1
```

```
console.log(button.getAttribute("name"));  // submitBtn
```

```
console.log(button.getAttribute("value")); // Send
```

```
const button = document.getElementById("btn1");
```

```
// Set new attributes
```

```
button.setAttribute("name", "submitBtn");
```

```
button.setAttribute("value", "Send");
```

```
button.setAttribute("class", "primaryBtn");
```


CreateElement() + appendChild



Used to create and add a new element at the end.

```
<div id="container">
```

```
    <p>Existing paragraph</p>
```

```
</div>
```

```
<script>
```

```
    // Step 1: Create a new element
```

```
    let newParagraph = document.createElement("p");
```

```
    // Step 2: Add text content
```

```
    newParagraph.textContent = "I am a new paragraph  
added with JavaScript!";
```

```
    // Step 3: Append to the parent div
```

```
    document.getElementById("container").appendChild(newPa  
ragraph);
```

```
</script>
```

What Happens?

- `createElement("p")`
→ Creates a `<p>` element.
- `textContent` → Adds text inside it.
- `appendChild(newParagraph)` → Adds it inside `<div id="container">`.

createTextNode/createAttribute



We create a new text node (called **newText**) using the **createTextNode()** method. This will contain whatever text was found in **textarea**.

// Create a new paragraph element

```
let newParagraph = document.createElement("p");
```

// Create a text node

```
let text = document.createTextNode("Hello, this is a text node!");
```

// Create an ID attribute

```
let idAttr = document.createAttribute("id");
```

// Assign a value to the attribute

```
idAttr.value = "myButton";
```

// Append the text node to the paragraph

```
newParagraph.appendChild(text);
```

// Attach the attribute to the button

```
newParagraph.setAttributeNode(idAttr);
```

// Append the paragraph to the body

```
document.body.appendChild(newParagraph);
```

Adds an Element Before Another



The `insertBefore()` method inserts an element **before a specific existing child**.

```
parentElement.insertBefore(newElement, existingChild);
```

Example: Inserting a New Item Before Another

```
// Select the parent
let parent = document.getElementById("container");

// Create a new element
let newItem = document.createElement("p");
newItem.textContent = "This is inserted before an existing item!";

// Select an existing child
let existingItem = document.getElementById("oldItem");

// Insert before oldItem
parent.insertBefore(newItem, existingItem);
```

The new `<p>` is added before `#oldItem`.



Remove Child

```
let parent =  
document.getElementById("container");  
let child =  
document.getElementById("item");  
parent.removeChild(child);
```

- ▶ Finds the parent element with ID "container"
- ▶ Removes the child element with ID "item"

Delete last paragraph on the page



```
var allGrafts = document.getElementsByTagName("p");
```

getElementsByTagName method collect all the paragraph tags in our page and store them in the **allGrafts** array.

```
if (allGrafts.length > 1){  
    var lastGraf = allGrafts[allGrafts.length-1];  
    var docBody = document.getElementsByTagName("body")[0];  
    docBody.removeChild(lastGraf);  
}
```

In order to modify the document we need to get the content of the **body**.

Once we have got that, its simply a matter of calling the

docBody.removeChild() method and passing it **lastGraf**.

Which tells JavaScript which paragraph we want to delete. Our page should immediately show one less paragraph

Removing an Element on Click of a button



```
<div id="container">
  <p id="text">Click the button to remove me!</p>
  <button onclick="removeElement()">Remove</button>
</div>

<script>
  function removeElement() {
    let element = document.getElementById("text");
    if (element) {
      element.remove(); // Removes the paragraph
    }
  }
</script>
```

Replacing Node



```
<div id="parent">
  <p id="oldChild">This is the old paragraph.</p>
</div>

<button onclick="replaceElement()">Replace Child</button>

<script>
  function replaceElement() {
    let parent = document.getElementById("parent");
    let oldChild = document.getElementById("oldChild");

    // Create a new element
    let newChild = document.createElement("p");
    newChild.textContent = "This is the new paragraph.";

    // Replace the old child with the new one
    parent.replaceChild(newChild, oldChild);
  }
</script>
```



JQUERY



Introduction to JQuery

- What is jQuery?
 - A fast, small, and feature-rich JavaScript library.
 - Simplifies HTML document traversal, event handling, and animation.
 - Ensures cross-browser compatibility.
- Why use jQuery?
 - Concise and easy-to-use syntax.
 - Reduces the need for complex JavaScript code.
 - Widely supported and used in web development.



Adding JQuery

- Ways to include jQuery:
 - Download from jquery.com and include locally.
 - Use a CDN (e.g., Google or jQuery CDN):

```
<script  
  src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```
- Placing the script in your HTML file:
 - Inside the `<head>` or before closing `<body>`.
 - Recommended: Place it before `</body>` for better performance.



Basic Syntax

Structure:

```
$(document).ready(function() {  
    // jQuery code here  
});
```

- `$` is the jQuery selector.
- `$(document).ready()` ensures the DOM is fully loaded before executing scripts.



JQuery Selector

Select elements like CSS selectors:

- `$("p").hide();` // Hides all `

` elements
- `$("#myId").show();` // Shows the element with ID 'myId'
- `$(".myClass").fadeOut();` // Fades out elements with class 'myClass'

jQuery Selectors

- Few samples for the selectors

Syntax

`$(this)`

`$("p")`

`$("p.intro")`

`$(".intro")`

`$("#intro")`

`$("ul li:first")`

`$("[href$='.jpg']")`

`$("div#intro
.head")`

Description

Current HTML element

All `<p>` elements

All `<p>` elements with `class="intro"`

All elements with `class="intro"`

The first element with `id="intro"`

The first `<li>` element of each `<ul>`

All elements with an `href` attribute that ends with `".jpg"`

All elements with `class="head"` inside a `<div>` element with `id="intro"`

jQuery Attributes

- Read
 - `$(#image).attr("src");`
- Set
 - `$(#image).attr("src", "images/jquery1.jpg");`
- Multiple Set
 - `$(#image).attr({src: "images/jquery1.jpg", alt: "jQuery"});`
- Set Class
 - `$(p:last).addClass("selected");`
- Read/Set Html
 - `$(#id).html() & $(#id).html("value");`



Event handling in JQuery

jQuery makes event handling simple:

```
$("#btn").click(function() {  
    alert("Button clicked!");  
});
```

- Common events:
 - `click()`, `mouseover()`, `mouseleave()`,
`keydown()`, `submit()`



JQuery Effects & Animations

Hide/Show Elements:

```
$("#box").hide();
```

```
$("#box").show();
```

```
$("#box").toggle();
```

Fading Effects:

```
("#box").fadeIn();
```

```
$("#box").fadeOut();
```

Sliding Effects:

```
$("#box").slideUp();
```

```
$("#box").slideDown();
```


jQuery Fading Effects

- ▶ With jQuery you can fade elements in and out of visibility.
- ▶ Click to fade in/out panel

Examples:

- ▶ jQuery fadeIn()
Demonstrates the jQuery fadeIn() method.
- ▶ jQuery fadeOut()
Demonstrates the jQuery fadeOut() method.
- ▶ jQuery fadeToggle()
Demonstrates the jQuery fadeToggle() method.
- ▶ jQuery fadeTo()
Demonstrates the jQuery fadeTo() method.

jQuery Effects - Sliding

Examples

slideDown()

Demonstrates the jQuery slideDown() method.

slideUp()

Demonstrates the jQuery slideUp() method.

► slideToggle()

Demonstrates the jQuery slideToggle() method.



Manipulating HTML and CSS

Change content:

- `$("#text").text("New Content");`
- `$("#text").html("<b>Bold Content</b>");`

Change CSS:

- ▶ `$("#box").css("background-color", "blue");`

jQuery HTML Manipulation

- jQuery HTML Manipulation Methods

Function	Description
<code>\$(selector).html(content)</code>	Changes the (inner) HTML of selected elements
<code>\$(selector).append(content)</code>	Appends content to the (inner) HTML of selected elements
<code>\$(selector).after(content)</code>	Adds HTML after selected elements

jQuery CSS Manipulation

- **jQuery css() Method**

- `css(name)` - Return CSS property value
- `css(name,value)` - Set CSS property and value
- `css({properties})` - Set multiple CSS properties and values

- **Examples**

- Return CSS Property
 - `$(this).css("background-color");`
- Set CSS Property & value
 - `$("#p").css("background-color","yellow");`
- Set Multiple values in CSS
 - `$("#p").css({"background-color":"yellow","font-size":"200%"});`
- Size Manipulation
 - `$("#div1").height("200px"); $("#div2").width("300px");`